FILE: C:\HCL_TRAINING\server\src\controllers\reportController.js

LINES: 169 | SIZE: 5.3 KB | LANGUAGE: JS

MODULE: Funnel Drop-Off Analytics, Skill Distributions, and CSV Export

```
const ScreeningResult = require('../models/ScreeningResult');
const Job = require('../models/Job');
const Application = require('../models/Application');
const Candidate = require('../models/Candidate');
const { logActivity } = require('../services/auditLogger');

// @desc    Get aggregate screening & recruitment reports
// @route   GET /api/reports/summary
const getReportsSummary = async (req, res, next) => {
  try {
    const { jobId, dateRange } = req.query;
    const orgId = req.organizationId;

    // Filter
    const filter = { organization: orgId };
    if (jobId && jobId !== 'all') {
      filter.jobId = jobId;
    }

    // 1. Job-wise screening summary
    const jobs = await Job.find({ organization: orgId }).select('title department status
 createdAt');

    const jobSummaries = await Promise.all(
      jobs.map(async (j) => {
        const total = await Application.countDocuments({ jobId: j._id });
        const shortlisted = await Application.countDocuments({
          jobId: j._id,
          stage: { $in: ['Shortlisted', 'Interview', 'Offer', 'Hired'] },
        });
        const hired = await Application.countDocuments({ jobId: j._id, stage: 'Hired' })
;

        const avgScoreAgg = await ScreeningResult.aggregate([
          { $match: { jobId: j._id } },
          { $group: { _id: null, avgScore: { $avg: '$overallScore' } } },
        ]);

        return {
          jobId: j._id,
          title: j.title,
          department: j.department,
          status: j.status,
          totalCandidates: total,
          shortlistedCandidates: shortlisted,
          hiredCandidates: hired,
          avgMatchScore: avgScoreAgg.length > 0 ? Math.round(avgScoreAgg[0].avgScore) :
0,
        };
      })
    );

    // 2. Recruitment Funnel Analytics (stage drop-off rates)
    const funnelStages = ['New', 'Screening', 'Shortlisted', 'Interview', 'Offer', 'Hire
d'];
    const funnelData = await Promise.all(
      funnelStages.map(async (stageName) => {
        const count = await Application.countDocuments({
          ...filter,
          stage: stageName,
        });
        return { stage: stageName, count };
      })
    );

    // 3. Top Skills Distribution across candidates in the talent pool
```

```
63 |      const candidates = await Candidate.find({ organization: orgId }).select('skills');
64 |      const skillCounts = {};
65 |      candidates.forEach((c) => {
66 |        (c.skills || []).forEach((s) => {
67 |          skillCounts[s.name] = (skillCounts[s.name] || 0) + 1;
68 |        });
69 |      });
70 |
71 |      const skillDistribution = Object.entries(skillCounts)
72 |        .map(([name, count]) => ({ skill: name, count }))
73 |        .sort((a, b) => b.count - a.count)
74 |        .slice(0, 10);
75 |
76 |      // 4. Candidate Ranking List
77 |      const candidateRankings = await ScreeningResult.find(filter)
78 |        .sort({ overallScore: -1 })
79 |        .limit(50)
80 |        .populate('candidateId', 'name email location totalExperienceYears')
81 |        .populate('jobId', 'title department');
82 |
83 |      res.status(200).json({
84 |        success: true,
85 |        data: {
86 |          jobSummaries,
87 |          funnelData,
88 |          skillDistribution,
89 |          candidateRankings,
90 |        },
91 |      });
92 |    } catch (error) {
93 |      next(error);
94 |    }
95 |  };
96 |
97 |  // @desc    Export candidate rankings to CSV format
98 |  // @route   GET /api/reports/export/csv
99 |  const exportRankingsCSV = async (req, res, next) => {
100 |    try {
101 |      const { jobId } = req.query;
102 |      const filter = { organization: req.organizationId };
103 |      if (jobId && jobId !== 'all') filter.jobId = jobId;
104 |
105 |      const results = await ScreeningResult.find(filter)
106 |        .sort({ overallScore: -1 })
107 |        .populate('candidateId')
108 |        .populate('jobId');
109 |
110 |      // Build CSV string
111 |      const headers = [
112 |        'Candidate Name',
113 |        'Email',
114 |        'Job Title',
115 |        'Overall Match Score',
116 |        'Skill Alignment',
117 |        'Semantic Similarity',
118 |        'Experience Score',
119 |        'Qualification Score',
120 |        'Years Experience',
121 |        'Matched Required Skills',
122 |        'Missing Required Skills',
123 |      ];
124 |
125 |      const rows = results.map((r) => {
126 |        const c = r.candidateId || {};
127 |        const j = r.jobId || {};
128 |        const matched = (r.matchedRequiredSkills || []).map((m) => m.skill).join('; ');
129 |        const missing = (r.missingRequiredSkills || []).join('; ');
130 |
131 |        return [
132 |          `"${(c.name || 'Candidate').replace(/"/g, '""')}"`,
133 |          `"${(c.email || '').replace(/"/g, '""')}"`,
134 |          `"${(j.title || '').replace(/"/g, '""')}"`,
```

```
135 |           r.overallScore,
136 |           r.componentScores.skillAlignment,
137 |           r.componentScores.semanticSimilarity,
138 |           r.componentScores.experienceScore,
139 |           r.componentScores.qualificationScore,
140 |           c.totalExperienceYears || 0,
141 |           `"${matched.replace(/"/g, '""')}"`,
142 |           `"${missing.replace(/"/g, '""')}"`,
143 |         ].join(',');
144 |       });
145 |
146 |       const csvContent = [headers.join(','), ...rows].join('\n');
147 |
148 |       await logActivity({
149 |         organizationId: req.organizationId,
150 |         userId: req.user._id,
151 |         action: 'DATA_EXPORTED',
152 |         entityType: 'Settings',
153 |         details: { exportType: 'CSV_RANKINGS', rowCount: results.length },
154 |         req,
155 |       });
156 |
157 |       res.setHeader('Content-Type', 'text/csv');
158 |       res.setHeader('Content-Disposition', `attachment; filename=hiresense_candidate_ranki
    >   ngs_${Date.now()}.csv`);
159 |       return res.status(200).send(csvContent);
160 |     } catch (error) {
161 |       next(error);
162 |     }
163 |   };
164 |
165 |   module.exports = {
166 |     getReportsSummary,
167 |     exportRankingsCSV,
168 |   };
169 |
```